

NANOPITCH: ACCESSIBLE DEEP LEARNING EDUCATION FOR MUSIC INFORMATION RETRIEVAL

Anonymous Authors
Anonymous Affiliations
anonymous@ismir.net

ABSTRACT

Teaching deep learning for music information retrieval (MIR) presents significant challenges: GPU requirements, complex audio processing pipelines, and a persistent gap between theoretical instruction and practical experience. In this paper, we present NanoPitch, an open-source educational toolkit that addresses these challenges through hands-on *pitch estimation*. Students train a lightweight GRU-based model ($\sim 333K$ parameters) on a laptop CPU, experiment with data augmentation and architecture modifications, and deploy their trained model to a web browser via C and WebAssembly. We deployed the toolkit in a university course with 17 enrolled students, of whom 12 completed the project and all evaluation instruments. A pre/post knowledge quiz revealed improvement (60.9% vs. 75.6%, +14.7 percentage points), with the largest gains on concepts practiced hands-on. Self-efficacy ratings increased (3.80 vs. 5.00 on a 7-point scale, Cohen’s $d = 0.86$, large effect). All participating students achieved functional pitch estimation models with Raw Pitch Accuracy between 90.8% and 98.0%. Notably, 100% of students reported using AI coding tools, primarily for concept explanation rather than code generation. We present NanoPitch as a case study in accessible, end-to-end deep learning education for MIR.

1. INTRODUCTION

Deep learning has transformed music information retrieval, achieving state-of-the-art results in pitch estimation [1, 2], music transcription [3], and numerous other tasks. Yet teaching students to build, train, evaluate, and deploy deep learning systems for audio remains difficult. GPU compute costs create access barriers, audio processing pipelines require specialized knowledge of spectral representations and real-time constraints, and the gap between textbook theory and practical ML engineering is wide. Students who complete a deep learning course can often describe backpropagation and name common architectures, but few have trained a model on real data, diagnosed training failures using gradient norms and learning

curves, or deployed a model to run under real-time constraints on end-user hardware.

Existing educational resources address parts of this problem. Foundational textbooks cover digital signal processing [4, 5] and music processing [6], while deep learning curricula provide theoretical grounding in architectures and optimization [7, 8]. However, few resources give students end-to-end experience with the full deep learning lifecycle, from data preparation through model training, evaluation, and real-time deployment, on a genuine audio task, without requiring specialized hardware.

We bridge this gap with NanoPitch, an open-source educational toolkit in which students train a lightweight neural network for monophonic *pitch estimation*, a core MIR task with clear evaluation metrics and immediate perceptual feedback. We designed the system around a key principle: *minimize the gap between the educational project and real-world production ML*. The model architecture uses *causal convolutions* and GRU layers that process audio frame-by-frame, the same causal constraints faced in production real-time systems. The training pipeline uses pre-extracted features distributed via HuggingFace to eliminate GPU and library dependencies, reducing setup to a single `pip install`. The deployment pipeline exports trained weights from PyTorch to a C inference engine compiled to WebAssembly, mirroring industry workflows where models are prototyped in Python and deployed in lower-level languages. Students can see their model tracking their own voice in a browser, experiencing the accuracy-latency tradeoff directly through a *real-time factor* (RTF) meter.

We deployed the toolkit in a special topics university course during Spring 2026. Students worked independently with the repository over approximately two weeks, supported by two weekly class sessions that provided context on pitch estimation, deep learning, and audio processing. We present the toolkit design and report empirical results from this classroom deployment, framed as a case study rather than a controlled experiment.

The contributions of this work are: (i) an open-source, self-contained toolkit for teaching deep learning through pitch estimation, requiring only a laptop CPU; (ii) a pedagogical design that spans the full ML lifecycle from feature extraction to browser deployment, with six structured experimental directions; and (iii) empirical evidence from a classroom deployment demonstrating measurable learning gains in both knowledge and self-efficacy.



2. RELATED WORK

2.1 Pitch Estimation

Monophonic pitch estimation, determining the fundamental frequency (f_0) of a single voice or instrument over time, has a long history in audio signal processing. Classical approaches include autocorrelation-based (YIN [9]), probabilistic (pYIN [10]), and formant-based (Praat [11]) methods. Deep learning has substantially advanced the field through CNN-based (CREPE [1]), fully convolutional (FCNF0 [12]), self-supervised (SPICE [13], PESTO [2]), and end-to-end (PENN [14], DeepF0 [15]) approaches. These systems prioritize state-of-the-art accuracy, typically employing large models or complex training procedures. NanoPitch does not aim to compete with them; instead, we use a deliberately lightweight GRU-based architecture ($\sim 333\text{K}$ parameters) designed to be trainable on a laptop CPU and comprehensible to students encountering deep learning for the first time.

2.2 AI and ML in Education

Artificial intelligence is increasingly shaping both *how* students are taught and *what tools* students use while learning. Broad surveys document the integration of AI into educational settings across disciplines [16–18], while recent work examines the specific impact of large language models on engineering and technical education [19, 20]. A growing body of literature recognizes that students now routinely use AI tools, including ChatGPT, GitHub Copilot, and Claude, as learning companions during technical coursework, raising questions about how to design assignments that remain pedagogically effective in an AI-assisted learning environment.

The curriculum design closest to NanoPitch is the deep learning audio syllabus of Tahiroğlu et al. [21], which introduces deep learning concepts through music composition and production. NanoPitch differs in several respects: it focuses on a single, well-scoped task (pitch estimation) with quantitative evaluation metrics, includes real-time deployment as a learning objective, and provides structured experimental directions that guide students through the iterative process of ML engineering.

2.3 MIR and DSP Education

Fundamentals of Music Processing [6], *DSP First* [4], and *Think DSP* [5] provide pedagogical treatments of MIR algorithms and digital signal processing with hands-on exercises. Flipped classroom approaches have been applied to signal processing education with positive results [22, 23]. Recent ISMIR tutorial books introduce deep learning architectures and training paradigms for MIR tasks [24, 25]. However, these resources primarily teach signal processing and MIR theory or address individual training subtasks; they do not include hands-on deep learning model training, hyperparameter tuning, or deployment to a real-time target platform. NanoPitch complements these foundational resources by providing a structured, self-contained project

that takes students from spectral features through neural network training to real-time browser inference.

3. SYSTEM DESIGN

3.1 Architecture

NanoPitch takes a 40-band log-mel spectrogram as input, computed at 16 kHz with a 25 ms Hann window and 10 ms hop size. The architecture, shown in Figure 1, consists of three processing stages followed by dual output heads.

Feature extraction. Two causal Conv1d layers (kernel size 3, 96 output channels) with LayerNorm [26], LeakyReLU [27] activation, and Dropout [28] extract local spectral features. Causal (left) padding ensures that the output at time t depends only on inputs at times $t-2$, $t-1$, and t , never on future frames. This is essential for real-time streaming inference: when a new audio frame arrives, the model can produce an output without waiting for subsequent frames.

Temporal modeling. Three GRU [29] layers with 96 hidden units each model temporal dependencies across frames. A *skip connection* adds the output of the first GRU layer to the output of the third, improving gradient flow during training and allowing the network to combine short-range and longer-range temporal context. We chose the GRU over the LSTM [30] for its simpler gate structure (two gates vs. three), which makes the architecture easier for students to understand and reduces the parameter count.

Output heads. Two independent linear heads produce the model’s predictions. The *voice activity detection* (VAD) head outputs a single sigmoid score representing the probability that the current frame contains singing. The pitch head outputs 360 independent sigmoid scores, one per 20-cent pitch bin spanning approximately B0 (31.7 Hz) to B6 (2006 Hz), spanning a range of six octaves. The use of independent sigmoids rather than a softmax allows the model to express graded uncertainty across adjacent bins, producing a smooth pitch *posteriorgram* that is decoded into a single f_0 estimate per frame via *Viterbi decoding*.

The full model contains approximately 333K trainable parameters. At the default GRU size of 96 units, inference runs at RTF 0.1–0.3 in WebAssembly on consumer laptops, well within the real-time budget (RTF < 1.0). Students can experiment with larger GRU sizes (e.g., 128 or 256 units) to observe the accuracy-latency tradeoff directly: a 256-unit GRU ($\sim 1.6\text{M}$ parameters) achieves higher accuracy but may exceed RTF 1.0, causing audible frame drops in the browser.

3.2 Training Pipeline

A central design decision is the use of *pre-extracted features*. Clean vocal mel spectrograms and pitch annotations (extracted from studio-recorded singing across 9 languages) and noise mel spectrograms (from environmental sound recordings) are distributed as compressed NumPy archives via HuggingFace. This eliminates the need for on-the-fly audio loading and feature computation, removing dependencies on librosa, torchaudio, or ffmpeg at train-

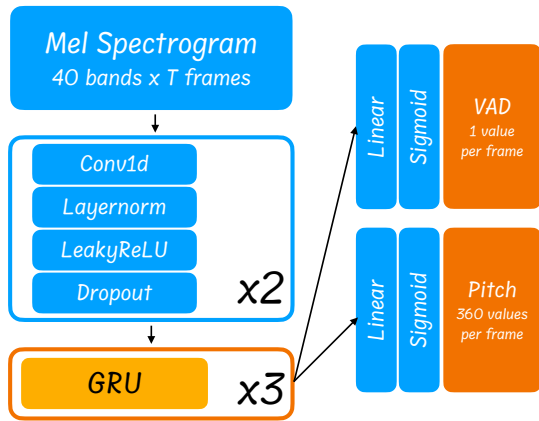


Figure 1. NanoPitch architecture. Audio is converted to a 40-band mel spectrogram, processed by two causal Conv1d layers and three GRU layers with a skip connection, then decoded into VAD and pitch posteriorgram outputs. The model contains $\sim 333\text{K}$ parameters.

ing time and reducing the setup burden to `pip install torch numpy scipy tqdm`.

The training loop uses `BCEWithLogitsLoss` for the VAD head and a VAD-weighted BCE loss for the pitch posteriorgram. The VAD weighting masks the pitch loss to zero on unvoiced frames, ensuring the model only receives pitch supervision when a voice is present. This design reflects a real-world consideration: predicting pitch during silence is meaningless, so the loss function should not penalize arbitrary predictions on silent frames. We use AdamW [31] with cosine annealing learning rate scheduling for optimization.

We provide students with a starter training script containing a stub augmentation function that returns clean mel spectrograms without modification. Implementing noise mixing in the log-mel domain is the first substantive coding task. A critical subtlety is that naive addition in log-power space ($\text{mel}_{\text{mix}} = \text{mel}_{\text{clean}} + \text{mel}_{\text{noise}}$) corresponds to multiplying spectral envelopes in the linear domain rather than adding acoustic signals. The correct operation is `logaddexp`, which computes $\log(\exp(a) + \exp(b))$ in a numerically stable manner.

The repository README describes six structured experimental directions: (1) log-mel noise augmentation and *SpecAugment* [32], (2) voiced/unvoiced class imbalance handling via positive weighting or focal loss, (3) context window length for GRU training, (4) extended training with checkpoint resumption, (5) gradient norm monitoring and learning rate scheduling, and (6) curriculum training with sequentially frozen output heads. Each direction includes implementation guidance, relevant literature, and expected outcomes, allowing students to choose experiments matching their interests and skill level.

3.3 Evaluation

We evaluate trained models using a held-out test set containing clean singing mixed with environmental noise at

six conditions: clean, +20, +10, +5, 0, and -5 dB SNR. This multi-condition evaluation teaches students about model robustness: a model trained only on clean data may achieve high accuracy in quiet conditions but degrade rapidly with increasing noise, while augmented models show a flatter degradation curve.

The evaluation uses a *realtime* Viterbi decoder with no lookahead, matching the exact decoding strategy used in the browser deployment. This decoder processes frames causally, emitting a pitch estimate for each frame as it arrives, without the ability to revise past estimates. We also provide an offline Viterbi decoder (which processes the full sequence before backtracing) as an upper bound, allowing students to observe the gap between causal and non-causal decoding.

Reported metrics include Raw Pitch Accuracy (RPA: fraction of voiced frames within 50 cents of ground truth), Raw Chroma Accuracy (RCA: same but ignoring octave errors), VAD Accuracy, Voicing Detection Rate (VDR: recall for voiced frames), Gross Error Rate, and Median Cents Error. The multi-metric evaluation exposes students to a key ML engineering challenge: optimizing for one metric (e.g., RPA) may degrade another (e.g., VDR), requiring thoughtful tradeoffs.

A competitive leaderboard, ranked by RPA on clean audio, provides motivation and enables students to compare strategies. The leaderboard uses the realtime decoder to ensure that rankings reflect deployable model quality.

3.4 Deployment Pipeline

The deployment pipeline comprises three stages. A Python export script serializes all model weights to a JSON format readable by a C inference engine. A C implementation reimplements the complete forward pass, including mel spectrogram computation, GRU cell updates, and Viterbi decoding, and is compiled to WebAssembly via Emscripten. Finally, a web application captures microphone audio via the Web Audio API, feeds frames to the WASM module, and renders a live pitch track alongside an RTF meter. Students encounter deployment-specific challenges along the way: ensuring numerical consistency between PyTorch and C, understanding why interpreted languages are too slow for real-time audio, and observing how model size directly impacts user experience through the RTF meter.

4. CLASSROOM DEPLOYMENT

We deployed NanoPitch in a special-topics course at a university (anonymized for review) during Spring 2026. The course covered topics in music technology and audio processing. Of 17 enrolled students (a mix of 12 undergraduate and 5 graduate levels), 12 completed the project and all evaluation instruments. No formal machine learning prerequisites were required; students had varying backgrounds in programming and signal processing.

The project spanned approximately two weeks of independent work. Two weekly class sessions provided context

on pitch estimation, deep learning fundamentals, and audio processing concepts, but the primary learning vehicle was independent experimentation with the repository. Students were expected to implement noise augmentation, run training experiments, evaluate their models, and optionally deploy to the browser. A competitive leaderboard accepted model submissions throughout the project period and was visible to all students.

Evaluation instruments. We administered three instruments. (1) A pre/post knowledge quiz containing 13 multiple-choice and 5 short-answer questions covering mel spectrograms, causal convolutions, GRU hidden states, loss functions, gradient dynamics, real-time deployment, and experimental design. The multiple-choice questions were designed to test understanding at Bloom’s taxonomy levels of comprehension through application. (2) A pre/post *self-efficacy* survey with 12 competency dimensions (e.g., “Train a neural network and interpret training curves,” “Diagnose model underperformance via training dynamics”) rated on a 1-7 Likert scale from “Not at all confident” to “Extremely confident.” (3) A post-project qualitative questionnaire including open-ended reflections on learning outcomes, AI tool usage patterns, and what students would do differently. IRB exemption was obtained for this evaluation.

5. RESULTS

5.1 Knowledge Quiz

Full cohort. Overall multiple-choice accuracy improved from 60.9% (95/156) to 75.6% (118/156), a gain of +14.7 percentage points across the full cohort ($N = 12$ pre, $N = 12$ post). As shown in Table 1, the three questions with the largest gains (+25.0 pp each) addressed: (1) combining log-power spectrograms using the `logaddexp` operation, which students practiced when implementing noise augmentation; (2) interpreting gradient norm spikes, which students observed in TensorBoard during training; and (3) understanding the consequence of $\text{RTF} > 1.0$ for real-time deployment, which students experienced when deploying models in-browser. Notably, all three of the largest gains correspond to concepts that students practiced hands-on during the project.

Matched pairs. Among 9 matched pairs (students who completed both pre and post quizzes), accuracy improved (66.7% vs. 75.2%, +8.5 pp). Six of 9 matched students improved, 2 maintained perfect scores (100% pre and post), and 1 declined. The declining student submitted both quizzes within 23 minutes on the same day, suggesting the “pre” response was not a true baseline. Table 1 presents per-question results for the full cohort.

Short-answer quality. Post-quiz responses also improved qualitatively. Responses to a question about designing a SpecAugment experiment consistently described proper ablation methodology, training two matched models with and without augmentation and evaluating on the same held-out test set, whereas pre-quiz responses were often vague or described uncontrolled comparisons (e.g.,

Topic	Pre	Post	Δ
Log-domain mixing	58.3	83.3	+25.0
Gradient spike interpretation	58.3	83.3	+25.0
RTF deployment consequence	50.0	75.0	+25.0
Why mel frequency scale	75.0	91.7	+16.7
GRU hidden state information	41.7	58.3	+16.7
VAD accuracy calculation	75.0	91.7	+16.7
Validation plateau next step	66.7	83.3	+16.7
RTF calculation	66.7	83.3	+16.7
Closing clean/noisy gap	66.7	83.3	+16.7
Mel spectrogram frame count	66.7	75.0	+8.3
Minimum latency (25 ms)	50.0	58.3	+8.3
Causal convolution definition	83.3	83.3	0.0
Doubling GRU size effects	50.0	41.7	−8.3

Table 1. Per-question accuracy (%) for the full cohort ($N=12$ pre, $N=12$ post). Δ is the change in percentage points. The three largest gains correspond to concepts practiced during hands-on project work.

“singing with slight music in background”). Responses to a question about investigating a suddenly diverging training loss shifted from single-word answers (“Overfitting”) to more nuanced reasoning mentioning learning rate instability, checkpoint mismatch, exploding gradients, and the distinction between training loss and evaluation metrics. The number of “I don’t know” responses on the training divergence question decreased (3 vs. 1, pre vs. post). Across all five short-answer questions, total “I don’t know” responses among matched pairs fell from 14/45 (31.1%) to 11/45 (24.4%), with reductions concentrated on questions about experimental design and training diagnostics, the two topics where students had the most hands-on practice.

5.2 Self-Efficacy

Self-efficacy increased substantially across all 12 competency dimensions. Five matched students completed both pre and post surveys, and Table 2 presents per-dimension results. Overall mean self-efficacy increased (3.80 vs. 5.00 on the 7-point scale, +1.20, Cohen’s $d = 0.86$), a large effect by conventional standards. All 12 dimensions showed positive mean gains, and all 5 matched students showed net improvement across the full set of dimensions.

The largest gains appeared in practical, experiential competencies: diagnosing model underperformance via training dynamics (+2.20, $d = 2.01$), implementing data augmentation strategies (+1.80, $d = 1.21$), and designing controlled experiments (+1.60, $d = 0.88$). Notably, “diagnosing underperformance” was the only dimension where all 5 matched students improved. Conversely, the smallest gains appeared in declarative knowledge, namely explaining mel spectrograms (+0.40) and explaining results to a non-technical audience (+0.20), which are less directly practiced in the project. Table 2 presents per-dimension results.

Competency	Pre	Post	Δ	d
Diagnose underperformance	2.60	4.80	+2.20	2.01
Data augmentation	3.00	4.80	+1.80	1.21
Experimental design	3.80	5.40	+1.60	0.88
Modify NN architecture	3.60	5.00	+1.40	1.23
Train NN / interpret curves	4.20	5.40	+1.20	0.92
Deploy to browser	2.80	4.00	+1.20	1.43
Accuracy-speed tradeoff	4.00	5.20	+1.20	1.43
Read research papers	4.80	6.00	+1.20	0.81
Explain GRU processing	3.20	4.20	+1.00	0.53
Evaluate with metrics	4.80	5.80	+1.00	1.00
Explain mel spectrogram	4.40	4.80	+0.40	0.35
Explain to non-technical	4.40	4.60	+0.20	0.09
Overall	3.80	5.00	+1.20	0.86

Table 2. Self-efficacy results ($N=5$ matched pairs, 1-7 Likert). d = Cohen’s d . All dimensions show positive mean change. The largest gains are in practical competencies directly exercised during the project.

5.3 Leaderboard

All students who submitted to the leaderboard achieved functional pitch estimation models. The leaderboard received 17 submissions from 11 unique students, all evaluated with the realtime Viterbi decoder matching browser deployment conditions. Best-per-student RPA on clean audio ranged from 90.8% to 98.0% (mean 95.3%, median 96.7%, $\sigma = 2.6\%$). As shown in the left panel of Figure 2, across all 17 submissions, VAD accuracy ranged from 79.9% to 98.6% (mean 95.0%) and median pitch error ranged from 1.9 to 18.0 cents (mean 9.5 cents). The mean clean-to-noisy gap (RPA Clean minus RPA at 0 dB SNR) was 5.6 percentage points, indicating that most models maintained reasonable robustness under moderate noise.

Strategy analysis. We observed a clear relationship between the number of optimization strategies employed and leaderboard rank. The 5 baseline-only submissions (default hyperparameters with minimal or no augmentation) clustered in ranks 8-16, while the top 7 positions were held by submissions employing at least two distinct strategies. The top-ranked submission (98.0% RPA Clean, 96.1% RPA macro-average) combined noise augmentation, SpecAugment, cosine annealing, loss weight tuning, VAD label improvements, and a larger GRU (128 units). Among top-5 submissions, the most common strategies were learning rate scheduling (4/5), noise augmentation (3/5), and loss weight tuning (3/5).

Iterative improvement. Students who submitted multiple checkpoints demonstrated iterative improvement consistent with the project’s emphasis on experimental methodology. One student progressed from 93.8% to 98.0% RPA (+4.2 pp) between an initial baseline and a multi-strategy optimization. Another progressed through four submissions spanning 90.0% to 96.6%, initially struggling with complex multi-strategy approaches before finding success with a simpler, well-calibrated configuration, an instructive example of how systematic simplification can outperform untested complexity.

5.4 AI Tool Usage

All 12 post-quiz respondents (100%) reported using AI tools during the project. The most commonly used tools were Claude (including Claude Code and Cursor with Claude, 5 students) and ChatGPT (5 students), followed by Cursor IDE (2), Codex (2), GitHub Copilot (2), and Gemini (1); many students used multiple tools simultaneously. As shown in the right panel of Figure 2, the dominant usage pattern was *concept explanation and learning aid* (50% of students), followed by brainstorming experiment ideas (25%), code generation (25%), debugging (17%), code analysis (17%), paper reading and summarization (17%), and passive autocompletion (17%).

Student descriptions revealed a spectrum of AI engagement. One student described using AI to understand “how my model results were impacted by the changes I made as well as points of improvement/weakness in the changes I was making to VAD/pitch weighting,” treating the AI as an interpretive companion for experimental results. Another described a progression from guidance to delegation: “I started by asking it to help me suggest changes for the basic stuff, like noise augmentation and SpecAugment. However, I ended up with a pretty high RPA but low VDR ... so I ended up putting it in Agent mode and letting it write a lot of the code.” A third used AI “to read research papers fast, and compare industry standards to what we have, and also to find weakness in the code.”

6. DISCUSSION

Experiential learning. The strongest knowledge gains occurred on questions tied directly to hands-on activities: log-domain signal mixing (implemented during augmentation), gradient dynamics (observed in TensorBoard), and real-time deployment constraints (experienced in-browser). This pattern, where the largest improvements appear on concepts that students practiced rather than merely read about, is consistent with *experiential learning* principles. Student reflections reinforced this interpretation. One student noted: “Being able to visualize how a model’s understanding of pitch and VAD improved in realtime via the TensorBoard helped demonstrate why it’s important to apply certain parameters at particular times in the training process.” Another reflected: “In theory, adding noise should improve robustness, but in practice I saw metrics like VDR drop before improving when I increased augmentation.” The multi-metric evaluation was itself a source of learning: one student wrote, “learning about the trade-offs in performance between different metrics... such as having to consider how the model’s training as adjusted for RPA improvements might affect VAD/VDR was a challenging exercise.” The dominant theme in “what would you do differently” responses was a desire for more systematic experimentation, changing one variable at a time and tracking results carefully, suggesting that the project may have cultivated metacognitive awareness about experimental methodology. As one student reflected: “I would run my experiments in a more organized fashion. I often

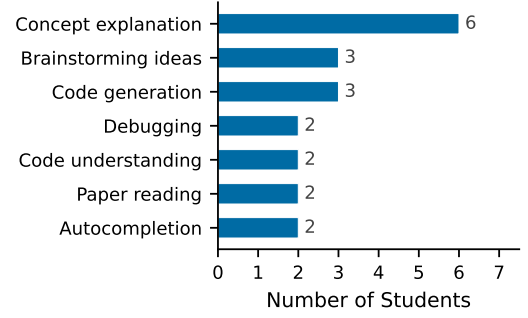
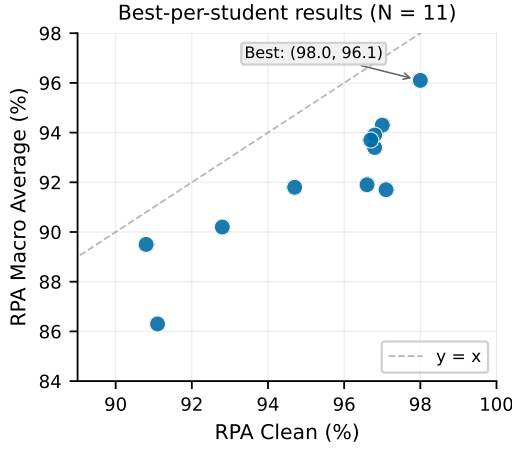


Figure 2. Left: Best-per-student RPA on clean audio ($N=11$ students). All students achieved functional models ($>90\%$ RPA). Right: AI tool usage categories ($N=12$). Concept explanation was the most common use case.

changed multiple parameters at a time instead of trying to isolate parameter impacts.”

AI-assisted learning. The 100% adoption rate of AI tools, with concept explanation as the primary use case, suggests that AI tools have become integral to how students learn ML. The spectrum of usage, from passive autocompletion to full agent-mode delegation, raises important questions for pedagogical design. One student’s account illustrates how AI and hands-on practice can reinforce each other: “I did not understand the idea of BCE with logits loss at first. . . I asked some questions to the AI and looked into the documentation to realise that it combining the sigmoid and bceloss, we get a basis on which we can do more operations like pos_weighting.” Even the student who used AI most aggressively (agent mode) demonstrated knowledge gains on the post-quiz and self-efficacy improvements, suggesting that learning occurred alongside delegation. NanoPitch’s structured experimental directions, multi-metric evaluation, and deployment requirement may create natural checkpoints that force engagement even when AI assists with implementation: students must still interpret whether their model improved and why. This points toward a design principle for the AI era: educational projects should include evaluation dimensions that require domain reasoning rather than relying solely on code correctness. Despite universal AI adoption, the strong knowledge and self-efficacy gains we observe suggest that AI tools complemented rather than replaced learning, though the absence of a control group prevents us from isolating their effect.

Limitations. This is a case study with a small sample ($N = 5$ -12 depending on instrument), no control group, and self-selection bias. Three of 9 matched quiz pairs submitted pre-quizzes on the same day as the post-quiz; restricting to the 6 clean pairs yields a smaller gain (+2.6 pp) with higher baselines (71.8% vs. 66.7%). Self-efficacy is self-reported and may reflect confidence rather than competence. Despite these limitations, the consistency of gains across instruments and their alignment with hands-on activities provide preliminary evidence of pedagogical effectiveness.

The 100% AI tool usage rate makes it impossible to isolate the toolkit’s contribution from AI assistance.

7. CONCLUSION

We present NanoPitch, an open-source educational toolkit that shows how accessible, hands-on deep learning for MIR can be delivered on laptop hardware with careful pedagogical design. By combining a lightweight trainable model, a concrete MIR task with clear evaluation metrics, and a deployment path aligned with common industry practice, the toolkit yielded measurable gains in conceptual knowledge (60.9% vs. 75.6%), self-efficacy (3.80 vs. 5.00, Cohen’s $d = 0.86$), and practical ML engineering skills. A central theme is that narrowing the distance between a classroom project and a production machine-learning workflow deepens learning, because it surfaces the tradeoffs and constraints that practitioners routinely face. All 11 students who used the leaderboard achieved deployable models, and trajectories of repeated submissions show students iterating in a way consistent with real experimental ML. As AI-assisted coding becomes standard, combining structured experimentation, multi-metric evaluation, and real-time deployment constraints, as in NanoPitch, may help keep project-based instruction substantive. The toolkit is released as open-source software.

8. AI USAGE STATEMENT

We used Claude Code, Codex, and Cursor to help develop the NanoPitch toolkit and to support the quantitative analyses in this paper. Students’ use of AI during the course project is summarized in Section 5.4.

9. ETHICS STATEMENT

The classroom evaluation was covered by an institutional IRB exemption. Survey and quiz records were de-identified before any analysis.

10. REFERENCES

- [1] J. W. Kim, J. Salamon, P. Li, and J. P. Bello, "CREPE: A convolutional representation for pitch estimation," in *ICASSP, IEEE International Conference on Acoustics, Speech and Signal Processing - Proceedings*, vol. 2018-April, 2018, pp. 161–165.
- [2] A. Riou, S. Lattner, G. Hadjeres, and G. Peeters, "PESTO: Pitch estimation with self-supervised transposition-equivariant objective," in *Proceedings of the 24th International Society for Music Information Retrieval Conference*. Milan: ISMIR, 2023, pp. 535–544.
- [3] E. Benetos, S. Dixon, Z. Duan, and S. Ewert, "Automatic music transcription: An overview," *IEEE Signal Processing Magazine*, vol. 36, no. 1, pp. 20–30, 2018.
- [4] J. H. McClellan, R. Schafer, and M. Yoder, *DSP First*, 2nd ed. Pearson, August 2015.
- [5] A. Downey, *Think DSP: digital signal processing in Python*. O'Reilly Media, Inc., 2016.
- [6] M. Müller, *Fundamentals of music processing: Audio, analysis, algorithms, applications*. Springer, 2015, vol. 5.
- [7] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *nature*, vol. 521, no. 7553, pp. 436–444, 2015.
- [8] I. Goodfellow, Y. Bengio, and A. Courville, *Deep learning*. MIT press, 2016.
- [9] A. de Cheveigné and H. Kawahara, "YIN, a fundamental frequency estimator for speech and music," *The Journal of the Acoustical Society of America*, vol. 111, no. 4, pp. 1917–1930, 2002.
- [10] M. Mauch and S. Dixon, "pYIN: A fundamental frequency estimator using probabilistic threshold distributions," in *2014 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, vol. 1, 2014, pp. 659–663.
- [11] P. Boersma, "Accurate short-term analysis of the fundamental frequency and the harmonics-to-noise ratio of a sampled sound," *IFA Proceedings 17*, vol. 17, pp. 97–110, 1993.
- [12] L. Ardaillon and A. Roebel, "Fully-convolutional network for pitch estimation of speech signals," in *Proceedings of the Annual Conference of the International Speech Communication Association, INTERSPEECH*, vol. 2019-September, 2019, pp. 2005–2009.
- [13] B. Gfeller, C. Frank, D. Roblek, M. Sharifi, M. Tagliasacchi, and M. Velimirovic, "SPICE: Self-supervised pitch estimation," *IEEE/ACM Transactions on Audio Speech and Language Processing*, vol. 28, pp. 1118–1128, 2020.
- [14] M. Morrison, C. Hsieh, N. Pruyne, and B. Pardo, "Cross-domain neural pitch and periodicity estimation," *CoRR*, vol. abs/2301.12258, 2023.
- [15] S. Singh, R. Wang, and Y. Qiu, "DeepF0: End-to-end fundamental frequency estimation for music and speech signals," in *ICASSP, IEEE International Conference on Acoustics, Speech and Signal Processing - Proceedings*, vol. 2021-June. Institute of Electrical and Electronics Engineers Inc., 2021, pp. 61–65.
- [16] M. Chassignol, A. Khoroshavin, A. Klimova, and A. Bilyatdinova, "Artificial intelligence trends in education: a narrative overview," *Procedia Computer Science*, vol. 136, pp. 16–24, 2018.
- [17] F. Ouyang, L. Zheng, and P. Jiao, "Artificial intelligence in online higher education: A systematic review of empirical research from 2011 to 2020," *Education and Information Technologies*, pp. 1–33, 2022.
- [18] O. Zawacki-Richter, V. I. Marín, M. Bond, and F. Gouverneur, "Systematic review of research on artificial intelligence applications in higher education—where are the educators?" *International Journal of Educational Technology in Higher Education*, vol. 16, no. 1, pp. 1–27, 2019.
- [19] E. Kasneci, K. Seßler, S. Küchemann, M. Bannert, D. Dementieva, F. Fischer, U. Gasser, G. Groh, S. Günnemann, E. Hüllermeier *et al.*, "Chatgpt for good? on opportunities and challenges of large language models for education," *Learning and Individual Differences*, vol. 103, p. 102274, 2023.
- [20] J. Qadir, "Engineering education in the era of ChatGPT: Promise and pitfalls of generative AI for education," in *2023 IEEE Global Engineering Education Conference (EDUCON)*. IEEE, 2023, pp. 1–9.
- [21] K. Tahiroğlu, S. Wang, E. Tampu, and J. Lin, "Deep learning with audio: An explorative syllabus for music composition and production," in *AIMC 2023 Proceedings: Conference on AI and Music Creativity 2023*, 2023, conference on AI and Music Creativity.
- [22] B. Van Veen, "Flipping signal-processing instruction [SP education]," *IEEE Signal Processing Magazine*, vol. 30, no. 6, pp. 145–150, 2013.
- [23] W. U. Bajwa, "On "flipping" a large signal processing class [SP education]," *IEEE Signal Processing Magazine*, vol. 34, no. 4, pp. 158–170, 2017.
- [24] M. Won, J. Spijkervet, and K. Choi, *Music Classification: Beyond Supervised Learning, Towards Real-world Applications*. <https://music-classification.github.io/tutorial>, 2021. [Online]. Available: <https://music-classification.github.io/tutorial>
- [25] G. Peeters, G. Meseguer-Brocal, A. Riou, and S. Lattner, *Deep Learning 101 for Audio-based MIR, ISMIR 2024 Tutorial*. San Francisco,

646 USA: [https://geoffroypeeters.github.io/deeplearning-](https://geoffroypeeters.github.io/deeplearning-101-audiomir-book)
647 101-audiomir-book, 2024. [Online]. Avail-
648 able: [https://geoffroypeeters.github.io/](https://geoffroypeeters.github.io/deeplearning-101-audiomir_book)
649 deeplearning-101-audiomir_book

650 [26] L. J. Ba, J. R. Kiros, and G. E. Hinton, “Layer normal-
651 ization,” *CoRR*, vol. abs/1607.06450, 2016.

652 [27] B. Xu, N. Wang, T. Chen, and M. Li, “Empirical evalu-
653 ation of rectified activations in convolutional network,”
654 *CoRR*, vol. abs/1505.00853, 2015.

655 [28] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever,
656 and R. Salakhutdinov, “Dropout: A simple way to
657 prevent neural networks from overfitting,” *Journal of*
658 *Machine Learning Research*, vol. 15, pp. 1929–1958,
659 2014.

660 [29] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bah-
661 danau, F. Bougares, H. Schwenk, and Y. Bengio,
662 “Learning phrase representations using RNN Encoder-
663 Decoder for statistical machine translation,” in *Confer-*
664 *ence on Empirical Methods in Natural Language Pro-*
665 *cessing (EMNLP)*, 2014.

666 [30] S. Hochreiter and J. Schmidhuber, “Long short-term
667 memory,” *Neural computation*, vol. 9, no. 8, pp. 1735–
668 1780, 1997.

669 [31] D. P. Kingma and J. L. Ba, “Adam: A method for
670 stochastic optimization,” in *3rd International Confer-*
671 *ence on Learning Representations, ICLR 2015 - Con-*
672 *ference Track Proceedings*, 2015.

673 [32] D. S. Park, W. Chan, Y. Zhang, C.-C. Chiu, B. Zoph,
674 E. D. Cubuk, and Q. V. Le, “SpecAugment: A simple
675 data augmentation method for automatic speech recog-
676 nition,” *arXiv preprint arXiv:1904.08779*, 2019.